

Tema2: Hashing

Docente : Héctor Xavier xLimón Riaño
email: xavier120@hotmail.com

Introducción

- ▶ El hashing es uno de los elementos principales de la seguridad criptográfica
- ▶ Involucra el concepto de **función de una sola vía y huella digital**
- ▶ Las funciones de hash sólo funcionan si cumplen las siguientes propiedades:
 - ▶ Producen valores únicos y repetibles (deterministas) para cada entrada
 - ▶ El valor de salida no da ninguna pista con respecto a la entrada que lo produjo

Introducción

- ▶ Algunas funciones de hash son mejores que otras cumpliendo las propiedades antes mencionadas
- ▶ Buenas: SHA-256, SHA-512
- ▶ No tan buenas: MD5, SHA-1. Éstas no deberían ser usada en producción

Cifrado vs Hashing

- ▶ El cifrado permite tener confidencialidad en los mensajes, pero no ayuda a verificar la integridad de los mismos
- ▶ Confidencialidad: sólo las personas autorizadas pueden leer el mensaje
- ▶ Integridad: las personas no autorizadas no pueden cambiar el contenido del mensaje sin que dicho cambio pueda ser percibido
- ▶ Sólo porque un mensaje esté cifrado (no pueda ser leído) no significa que no pueda ser alterado, incluso de formas que tengan sentido después de descifrar

Digests

- ▶ Su traducción en español sería resumen
- ▶ Una función de hash produce un resumen o huella digital de algún contenido binario
- ▶ Este resumen puede utilizarse como medida para verificación de tampering (alteraciones de información), utilizadas por ejemplo en análisis forense
- ▶ En una transmisión, la idea es transmitir el mensaje junto con su digest, para verificar que haya llegado sin alteraciones
- ▶ En realidad el asunto es más complejo, pero se verá en temas posteriores

hashlib

- ▶ python cuenta por defecto con una biblioteca llamada hashlib

```
1 import hashlib
2 md5hasher = hashlib.md5(b'cadenaBin')
3 md5hasher.hexdigest()
4 md5hasher.update(b'continuacionCadenaBin')
5 md5hasher.hexdigest()
```

Funciones de hash

- ▶ Una función de hash toma un contenido de cualquier tamaño (hasta vacío) y siempre produce un número de un tamaño fijo
- ▶ Por ejemplo, MD5 produce salidas de 16 bytes
- ▶ Este tamaño tiene una relación con la seguridad de la función, entre más pequeño es peor

Funciones de hash

Una vez más, las propiedades que una función de hash debe cumplir son

- ▶ El mismo documento siempre produce el mismo digest
- ▶ El digest parece aleatorio, si tienes el digest eso no te da pistas del documento de entrada,
- ▶ Idealmente cada documento con contenido diferente debería tener un digest diferente (lo cual en realidad no es posible)

Funciones de hash

- ▶ Los digests son como las huellas digitales: si tienes al sujeto se puede verificar que sea quien dice ser mediante su huella digital
- ▶ Sin embargo, si sólo se tiene la huella digital no es fácil determinar el sujeto que la posee
- ▶ Es fácil calcular el digest de un documento, pero si se tiene sólo el digest es muy difícil determinar el documento que lo produjo, entre más difícil mejor
- ▶ Por esta razón son llamadas funciones de una sola vía: se puede ir de la entrada a la salida, pero no al revés

Ejercicios

- ▶ Extiende el cifrador de César binario para que incluya una verificación de integridad mediante SHA-256
- ▶ El profesor demostrará cómo agregar el hash en el cifrado
- ▶ Tu tarea consistirá en verificar el hash al descifrar, lanzando una excepción si el archivo cifrado es alterado

Hashing

- ▶ En computación, el término función de hash casi siempre se refiere a una función criptográfica
- ▶ Sin embargo, el término es más general
- ▶ Por ejemplo una función de hash muy simple es la que establece si un número es par o impar
- ▶ Una función de hash esencialmente relaciona elementos de un dominio muy grande, con elementos de un codominio relativamente pequeño

Hashing

- ▶ El tamaño del codominio depende de la función de hash, por ejemplo, ya se dijo que MD5 genera digest de 16 bytes, ¿Cuál es el tamaño del codominio para MD5?
- ▶ Si se considera un número muy grande de documentos de entrada, es posible que muchos de ellos produzcan el mismo hash
- ▶ Dado lo anterior, las funciones de hash tienen **perdida**. Se pierde información de la fuente al digest
- ▶ Que tenga perdida es importante, de lo contrario habría una forma de regresar del digest al documento original

Hashing

- ▶ Considerando de nuevo la función par
- ▶ Cualquier número puede dar como resultado 0 (par) o 1 (impar)
- ▶ Sería imposible saber, dado un valor 0 o 1 que número produjo dicho resultado
- ▶ Esta función tiene todas las propiedades: tiene perdida, comprime la entrada y es consistente (dada una entrada siempre produce la misma salida)

Hashing

- ▶ Las propiedades de consistencia, compresión y pérdida son comunes en cualquier función de hash, pero no suficientes para ser seguras o criptográficas
- ▶ Se requieren las siguientes propiedades:
 - ▶ Resistencia de preimagen
 - ▶ Segunda resistencia de preimagen
 - ▶ Resistencia de colisiones

Resistencia de preimagen

- ▶ Una preimagen básicamente es el conjunto de entradas de una función de hash que producen el mismo valor de hash
- ▶ Por ejemplo, para la función de paridad, la preimagen para el valor 0 es un conjunto de números infinitos, todos aquellos que sean par

Resistencia de preimagen

- ▶ Otro ejemplo: el hash MD5 de la cadena hola es 4d186321c1a7f0f354b297e8914ab240
- ▶ Esto quiere decir que hola es un elemento de la preimagen de 4d186321c1a7f0f354b297e8914ab240
- ▶ Más formalmente: Una preimagen de una función de hash H y un valor de hash k es el conjunto de valores de x para los cuales $H(x) = k$

Resistencia de preimagen

- ▶ Si se piensa por un momento, al tener un digest, existe un número infinito de documentos que produjeron dicho digest
- ▶ Sin embargo, eso no quiere decir que dado un digest sea fácil encontrar un elemento de la preimagen
- ▶ La resistencia de preimagen es básicamente: si tengo un digest y no sé cómo se obtuvo, no puedo ni siquiera encontrar un elemento en la preimagen sin realizar una cantidad ridícula de trabajo computacional

Resistencia de preimagen

- ▶ Por ejemplo. Al ver el contenido del archivo shadow de un servidor Linux, el atacante puede ver sólo valores de hash, le debería resultar muy difícil (si se utilizaron buenas políticas de contraseñas) encontrar alguna cadena que produjo el hash

Resistencia de preimagen

- ▶ Idealmente, los elementos de la preimagen deberían estar muy separados entre ellos y no tener un espaciado predecible
- ▶ El hecho de que las funciones de hash tengan pérdida ayuda a la resistencia de preimagen
- ▶ Si una función de hash tiene buena resistencia de preimagen, los ataques en este sentido serán aleatorios o bien consistirán en enumeraciones muy largas (fuerza bruta) hasta encontrar una coincidencia, lo cual no es computacionalmente tratable

Resistencia de preimagen

- ▶ Al proceso de intentar encontrar un elemento de la preimagen dado un valor de hash se le llama invertir el hash
- ▶ Encontrar un inverso debería ser muy difícil
- ▶ Por ejemplo la función de paridad no tiene buena resistencia de preimagen y por lo tanto no es segura
- ▶ Pero MD5 por ejemplo es muy difícil de invertir, esto es, tiene buena resistencia de preimagen, dado que existen 2^{128} posibles valores de hash, es muy difícil encontrar una coincidencia
- ▶ Incluso si pudieras revisar 1 millón de valores de entrada que producen salidas diferentes, tomaría alrededor de 10^{26} años poder encontrar un valor de la preimagen

Segunda resistencia de preimagen

- ▶ Significa que si tu tienes al menos un documento que produce un hash conocido, sigue siendo muy difícil encontrar otro documento que produzca el mismo hash
- ▶ Por ejemplo. Imagina que vas a descargar un archivo de internet, y el que subió el archivo comparte públicamente el hash del mismo, para que los que descargan puedan comprobar que no hubo alteraciones después de descargar. Un atacante que modifica el archivo mientras está en tránsito, no será capaz de modificar el archivo de tal forma que haga coincidir el hash del contenido original
- ▶ Esto funciona porque no existe un patrón repetible en los elementos de la preimagen

Resistencia a colisiones

- ▶ Significa que es difícil encontrar dos entradas que produzcan el mismo hash
- ▶ No un hash específico sino cualquier valor de hash
- ▶ Por ejemplo, si en la escuela queremos encontrar dos personas que cumplen años exactamente el 5 de Febrero, las probabilidades son pequeñas, sin embargo, encontrar dos personas que cumplen años el mismo día, sin importar cual, es más fácil
- ▶ En general es más fácil encontrar una colisión en un hash cualquiera que encontrar un elemento de la preimagen de un hash dado

Resistencia a colisiones

- ▶ Por ejemplo. Imagina un gobierno que crea dos versiones del mismo software, una versión contiene software de espionaje para países enemigos y el otro es el software normal.
- ▶ El gobierno podría hacer que ambas versiones tuvieran el mismo valor de hash dada una pobre resistencia de colisiones

Resistencia a colisiones

- ▶ Una característica que ayuda a a resistencia de coaliciones es la propiedad de **avalancha**
- ▶ Esto es, ante cualquier cambio mínimo en un documento, el valor de hash cambia drásticamente y de forma impredecible, un mínimo del 50% de los bits del hash debería ser diferente

Seguridad en funciones de hash

- ▶ MD5 tiene vulnerabilidades de resistencia de colisiones, existiendo técnicas que permiten vulnerar este aspecto de forma relativamente eficiente. Esto permite ataques prácticos para por ejemplo obtener certificados falsos de TLS
- ▶ También se han encontrado vulnerabilidades en la resistencia de preimagen, siendo posible encontrar coincidencias en 2^{123} intentos, lo cual sigue siendo computacionalmente intratable
- ▶ Por esta razón nunca debe utilizarse MD5, es un algoritmo que ha caído en desuso desde hace 10 años
- ▶ De la misma forma SHA-1, que vino a sustituir a MD5, también tienen debilidades de resistencia a colisiones, y tampoco debe seguirse usando

Seguridad en funciones de hash

- ▶ Cuando un algoritmo tiene debilidades, aun cuando no sean debilidades prácticas, indica que puede tener otras aun no son descubiertas.
- ▶ Por ejemplo, en el caso de MD5, aunque no se hayan descubierto debilidades prácticas de resistencia de preimagen ni segunda resistencia de preimagen, el hecho de que se hayan descubierto otras vulnerabilidades en la misma, crea la sospecha de que existan otras no descubiertas
- ▶ Por esta razón se prefiere dejar de usar dichos algoritmos y desarrollar nuevos
- ▶ El consejo general actual es usar SHA-256 o bien SHA-512

Introducción

- ▶ Otro uso común de las funciones de hash es el almacenamiento de contraseñas
- ▶ Por ejemplo, en la bases de datos de cualquier aplicación web, lo más probable es que el password no se guarde directamente, sino el hash del mismo
- ▶ Los manejadores de bases modernos cuentan directamente con la funcionalidad de crear hashes, por ejemplo, en MariaDB (SHA-256): `select sha2('Maria', 256)`
- ▶ De comprometerse el contenido de la base de datos, las contraseñas no son conocidas directamente por el atacante

Proceso de validación de password

- ▶ Cualquier sistema que almacene el hash de un password realiza un proceso similar al que sigue para establecer si un password dado por un usuario es válido:
 1. Se computa el hash del password del usuario
 $H(\text{passwordUsuario})$
 2. Se compara con el hash almacenado para ese usuario, tal que
 $H(\text{passwordUsuario}) = \text{hashAlmacenado}$

Seguridad de passwords

- ▶ El hashing de passwords es en principio seguro, sin embargo, existen hashes que pudieran ser conocidos, como por ejemplo el hash de la palabra password (busca en google 5f4dcc3b5aa765d61d8327deb882cf99)
- ▶ Otro problema es que dos usuarios diferentes pueden estar usando el mismo password, al comprometer uno de los hashes, se comprometen todos
- ▶ La gente continua y continuará utilizando passwords tontos, debe buscarse la manera de aumentar la seguridad aun ante ese tipo de password
- ▶ Una solución puede ser la autenticación multi-factor, no usar códigos que se mandan por SMS (no es un canal seguro) o que se tenga que ver con preguntar cosas personales al usuario (mayor fricción). Mejor usar un token físico o bien aprovechar otra cuenta del usuario (correo, telegram, etc).

Seguridad de passwords

- ▶ La autenticación multi-factor no es apropiada en todas las situaciones, ya que generan fricción para el usuario
- ▶ Imagina que cada vez que quisieras entrar a facebook tuvieras que validar mediante un código que llega a tu correo electrónico
- ▶ Se debe buscar reforzar los hashes de contraseñas sin recurrir a la autenticación multi-factor
- ▶ No basta con utilizar un algoritmo de hash más seguro, por ejemplo en vez de MD5 usar SHA-256, aunque si debe hacerse
- ▶ El problema de fondo sigue siendo que las funciones de hash son deterministas, por lo tanto, dada una cadena conocida, siempre generan el mismo hash

Seguridad de passwords

- ▶ Los que se debe hacer es agregar un **salt**
- ▶ Un salt es un valor usualmente aleatorio que se agrega al password antes de aplicarse la función de hash
- ▶ El salt puede ser público o estar almacenado de forma plana, ver /etc/shadow. Usualmente el password hasehado y el salt plano se almacenan juntos
- ▶ Dada la característica de avalancha de las funciones de hash seguras, al agregar el salt, el hash cambiará drásticamente
- ▶ También al ser aleatorio, será muy poco probable que dos usuarios con el mismo password tengan el mismo salt
- ▶ Es muy importante no reusar salts, debe ser uno diferente por password, de lo contrario pierden sentido

Seguridad de passwords

- ▶ El salt ayuda particularmente a volver prácticamente inútiles los ataques de cracking de passwords conocidos como tablas arco iris
- ▶ En este tipo de ataques se tienen diccionarios muy grandes con passwords y hashes pre-computados, donde la recuperación de un password a través de un hash dado es muy eficiente
- ▶ Sin embargo, no previene otros ataques como son los ataques de diccionario o fuerza bruta

Seguridad de passwords

- ▶ En la práctica no basta sólo con agregar el salt directamente al password, se requieren otras técnicas de seguridad que varios algoritmos para la generación de contraseñas utilizan.
- ▶ Ejemplos de algoritmos de este tipo:
 - ▶ crypt
 - ▶ bcrypt
 - ▶ Argon2

crypt

- ▶ Lo utilizan por defecto los distros Linux actuales

```
1 import crypt
2 crypt.crypt('password', '$tipoHash$+Salt')
```

Script

```
1 import os
2 from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
3 from cryptography.hazmat.backends import default_backend
4 #generar hash seguro
5 salt = os.urandom(16)
6 kdf = Scrypt(salt=salt, length=32,
7             n=2**14, r=8, p=1,
8             backend=default_backend())
9 key = kdf.derive (b"my great password")
10
11 #Validar password
12 kdf = Scrypt(salt=salt, length =32,
13             n=2**14, r=8, p=1,
14             backend=default_backend())
15 kdf.verify(b"my great password", key)
16 print("Success! (Exception if mismatch)")
```

Ejercicios

- ▶ Dado un archivo shadow y un diccionario de palabras intenta encontrar todas las contraseñas asociadas
- ▶ Dada una contraseña hasheada mediante scrypt y un salt dado en base 64, crackea el password mediante fuerza bruta:
 - ▶ El password sólo tiene 3 caracteres
 - ▶ Los caracteres posibles son cualquier letra mayúscula o minúscula, sin incluir la ñ y cualquier dígito